

RWKV Paradigm

Migration Assessment

Migrating Large Models to RWKV

Paradigm Migration, Cost Model and Serving Economics

Not a proof of concept: this is a move from Transformer attention / KV-cache serving to recurrent-state / high-concurrency serving.

rwkv.cn

next-generation LLM
architecture
attention-free / no KV cache
constant speed / memory

PARADIGM

RADLADS

migration path

white-box full-size conversion

Case Anchor A

low-risk validation

KV-optimized / long-context model

Case Anchor B

high-upside validation

higher KV pressure / higher serving cost

2x-6x

serving upside

depends on concurrency and cache pressure

Decision: RWKV migration is a reusable architecture-migration framework for open-weight large models

Question	Recommendation	Reason	Confidence
Main method	RADLADS full-size white-box conversion	Preserves tokenizer, MLP/MoE and most knowledge path; replaces the sequence mixer	Medium
Case anchor A	KV-optimized long-context model with smaller active compute	Useful for validating architecture mapping, training stability and serving kernels; not a fixed model ranking	Medium
Case anchor B	Model with higher KV/cache pressure or higher serving cost	Useful for validating commercial cost-reduction upside after the method stabilizes	Medium-low
Flagship case	Very large, high-quality, long-context model	Best as a flagship validation after the joint migration path matures	Low
GAD role	Post-conversion alignment or smaller student	Helps instruction/coding-agent behavior; not architecture migration	Medium

One-line judgment

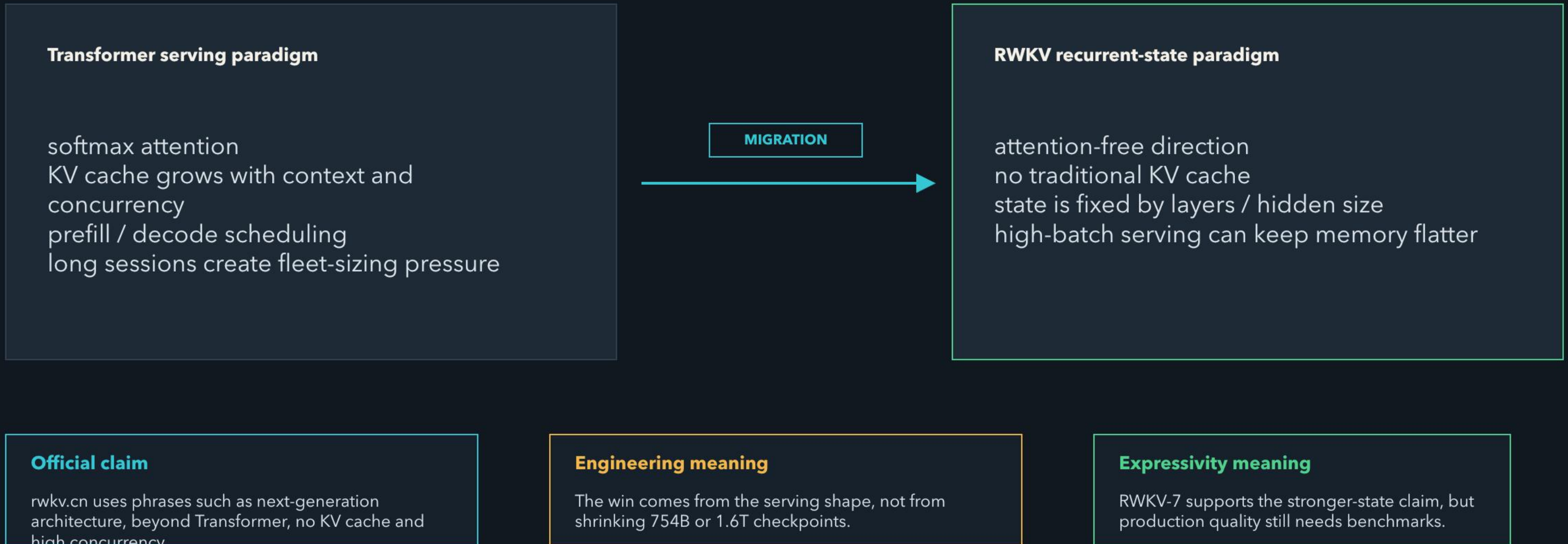
RADLADS provides the full-size architecture-migration path; GAD/SFT adapts behavior after migration.

Partner framing

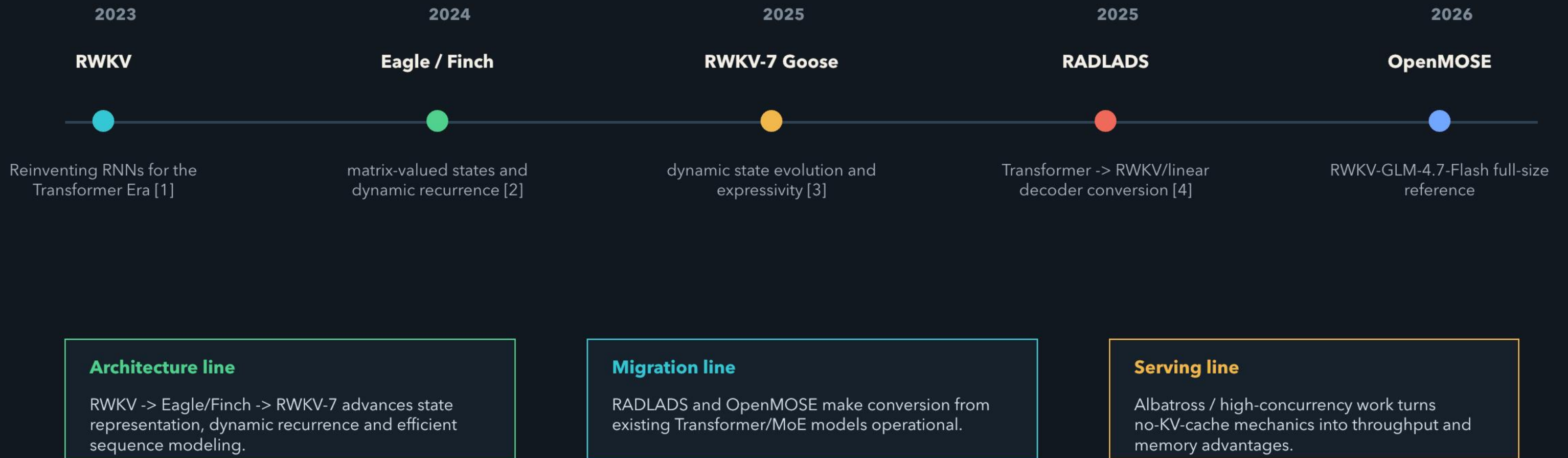
GLM and DeepSeek are public case anchors. The actual target should be selected by openness, KV/cache pressure, workload shape and quality gates.

Sources: RADLADS[4], GAD[5], public model cards and pricing docs.

rwkv.cn positions RWKV as an architecture paradigm, not a lab substitute



RWKV has grown from a single architecture into papers, engines, and migration tooling



The “more expressive than Transformer” claim should be tied to RWKV-7 dynamic state evolution

Fixed-depth Transformer

Attention is excellent for parallel matching, but fixed-depth Transformers face theoretical limits on some state-tracking and formal-language tasks.



RWKV-7 dynamic state

Input-dependent state transition matrices let each step update internal memory through recurrent computation.



Paradigm read

The paper supports stronger state expressivity in specific theoretical settings. It does not imply every benchmark is automatically better.

Recommended wording

RWKV-7 provides formal evidence for stronger dynamic state tracking than fixed-depth Transformers in certain formal-language settings.

Do not overclaim

Do not say RWKV naturally beats Transformer on all tasks. Converted GLM/DeepSeek models still need benchmark validation.

Basis: RWKV-7 “Goose” with Expressive Dynamic State Evolution [3].

RADLADS is full-size architecture migration; GAD is behavior distillation

Method	What it does	Needs weights	Full-size conversion	Best use	Main risk
RADLADS	Replaces attention with RWKV/linear mixer; aligns hidden states and logits	Yes	Yes	Open-weight Transformer/MoE migration	Memory, mapping, long-context regression
GAD	Black-box on-policy generation plus discriminator/reward distillation	No	No	Post-align or train smaller RWKV students	Behavior clone without internal parity
SFT / SeqKD	Supervised learning on teacher outputs	No	No	Format, style, cheap warmup	Surface imitation; weaker for agents/tools

Why RADLADS first

Case Anchor B and DeepSeek V4 are open weight, so weights, hidden states and logits are accessible.

Why GAD still matters

Conversion may lose instruction, coding-agent and tool-use style. GAD repairs behavior.

Boundary

GAD can train 7B/14B/30B RWKV students, but that is not full-size paradigm migration.

Full-size RWKV migration does not shrink the model; the win is serving state and scheduling

Target	Original size	Expected RWKV size	Size delta	What changes
OpenMOSE GLM-4.7 reference	31.221B	30.315B	-2.9% actual	attention -> RWKV-7/TICA
Case Anchor B	753.864B	~730B-760B	usually +/-5%	DSA/attention -> RWKV mixer
DeepSeek V4 Flash	284B/292B	~280B-295B	usually +/-5%	CSA/HCA -> RWKV mixer
DeepSeek V4 Pro (flagship case)	1.6008T	~1.55T-1.62T	usually +/-5%	flagship full-size project

Economic logic

Do not expect 754B or 1.6T to become small. The goal is to remove long-context KV-cache growth per request.

Research logic

Preserve MoE/MLP/embedding/tokenizer while avoiding full RWKV pretraining from scratch.

Source: OpenMOSE model card and prior safetensors parameter checks.

The inventory shows a selection method: different large models create different migration upside and engineering risk

Model	Open status	HF/card size	Active	Context	Architecture facts	RADLADS feasibility
OpenMOSE RWKV-GLM-4.7	open	30.315B	A3B	202K	47 RWKV-7 layers, 20 TICA layers	alpha reference exists
Case anchor B: GLM-5.1	open / MIT	753.864B	not stated	200K	78 layers, hidden 6144, 256 experts, top-8	high upside, high risk
Case anchor A: DeepSeek V4 Flash Base	open / MIT	292B/card 284B	13B	1M	43 layers, hidden 4096, 256 experts, top-6	low-risk case anchor
Flagship case: DeepSeek V4 Pro Base	open / MIT	1.6008T/card 1.6T	49B	1M	61 layers, hidden 7168, 384 experts, top-6	flagship case anchor

Partner evaluation start

Establish a shared evaluation protocol: same prompts, tokenizer, decoding, quality, long context, VRAM and throughput.

Low-risk example

Models with smaller active compute and long context are useful low-risk examples for validating the migration path and serving gain.

High-upside example

Models with higher serving cost or heavier KV/cache pressure are better for testing the commercial upside after the method is stable.

RADLADS replaces attention first, then pulls the student back with hidden states and logits



Why cheaper than pretraining

The new sequence mixer learns to mimic teacher attention outputs and logits while knowledge remains mostly in MLP/MoE paths.

Why still hard

GLM/DeepSeek are MoE long-context models; routing, normalization, positional logic and teacher-student memory can break the run.

The conversion-budget risk is not one GPU-hour number; it is failed runs and long-context debugging

Target	Hardware assumption	Wall clock	GPU-hours	compute-only	Project budget
DeepSeek V4 Flash -> RWKV	32-64 H100	2-5 days	1,536-7,680	\$6K-\$47K	\$50K-\$150K
Case Anchor B -> RWKV	128-256 H100	4-10 days	12,288-61,440	\$49K-\$378K	\$250K-\$1M
DeepSeek V4 Pro (flagship case) -> RWKV	256-512 H100	7-21 days	43,008-258,048	\$172K-\$1.59M	\$1M-\$4M

Estimate anchor

RADLADS published 7B/32B/72B dense conversion anchors; these ranges expand for MoE scale and retries.

Why wide ranges

Published numbers are not 284B/754B/1.6T MoE conversions. Real cost depends on mapping and stability.

Budget scope

compute-only excludes engineering, evaluation, data, failed runs and serving-kernel work.

H100 planning range: \$3.99-\$6.16 per GPU-hour.

Large models with existing KV compression can still migrate, but savings must be recalculated by concurrency and long sessions

Item	existing KV-compressed model	After RWKV migration	Interpretation
Single request, short context	KV pressure is already low	Limited savings	Active compute, weights and MoE routing still dominate
1M long context	KV is a small fraction of a common GQA-8 baseline	State is closer to fixed	Existing KV compression already removes part of the Transformer pain
500+ concurrent long sessions	Even small KV multiplies by users	State scales with concurrency but not context length	This is still the main source of high-concurrency RWKV savings for this model class



Conclusion

For KV-optimized models, RWKV is not a simple 10x story; a high-concurrency 2x-4x target is more realistic.

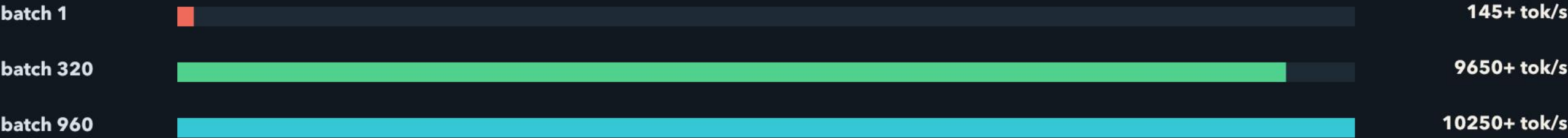
Contrast

For models with higher KV/cache pressure, RWKV savings can be closer to 3x-6x.

Basis: public reports/model cards; engineering estimate, not a provider quote.

RWKV's serving advantage is not single-stream speed; it is aggregate decode at high batch

Model	Precision	Batch	Aggregate speed	Stage	Hardware	vs batch 1
RWKV-7 7.2B	fp16	1	145+ tok/s	Decode	RTX5090 single	1x
RWKV-7 7.2B	fp16	320	9650+ tok/s	Decode	RTX5090 single	~66.6x
RWKV-7 7.2B	fp16	960	10250+ tok/s	Decode	RTX5090 single	~70.7x
RWKV-7 7.2B	fp16	1	11289+ tok/s	Prefill	RTX5090 single	N/A



Use boundary

This is mechanism evidence, not a linear extrapolation from 7.2B to 284B/754B. Measure the target MoE directly.

High concurrency is not only batching: DPLR lets RWKV-7 parallelize over time

Sequential recurrence

Token-by-token update:
 $S_t = P_t S_{t-1} + k_t^T v_t$

Low-latency decode remains sequential; training / long-prefill is time-dependent.

Explicit DPLR transition

$$P_t = \text{diag}(\exp(g_t)) + b_t a_t^T$$

Diagonal decay controls forgetting; low-rank terms add cross-dimensional coupling.

Chunk-wise affine

A chunk can be summarized as:
 $S' = M S + B$

Chunk-level (M,B) pairs compose via prefix scan / CP parallelism.

Why it matters

Long-sequence work moves from pure step-by-step recurrence into local chunk computation plus inter-chunk affine scan.

Cost

DPLR chunking adds intermediate tensors and HBM pressure; it mainly helps training / long prefill, not instant decode.

Cost-model impact

Separate three gains: high-batch decode, time-dimension prefill parallelism, and long-session state/KV memory.

New reference [9]: Zhiyuan Li, DPLR Mathematics: Explicit Transition Matrix Parallel Computation, 2026-02-21, zhiyuan1i.github.io/posts/dplr-mathematics/

High concurrency turns KV cache from a request-level issue into a fleet-sizing issue

Workload	Low-KV case -> RWKV	Flagship case -> RWKV	High-KV-pressure case -> RWKV	Reason
Short chat / <8K / low concurrency	5%-15%	5%-15%	10%-20%	Not enough KV pressure; active compute dominates
Normal coding / 8K-32K	10%-25%	10%-25%	20%-40%	Some batching and cache benefit
Agent service / 32K-128K	30%-55%	25%-50%	45%-70%	Concurrent sessions amplify state cost
Multi-agent fan-out	40%-70%	35%-65%	55%-80%	Closest to RWKV high-batch benefit
Long-lived high-concurrency context	50%-75%	45%-70%	65%-85%	KV/cache multiplies by session count

Low-KV case

With small KV already in place, the realistic target is not 10x; it is roughly 2x-4x at high concurrency.

High-KV-pressure case

With heavier KV/cache pressure, high-concurrency long-context savings can plausibly reach 3x-6x.

If quality holds and high concurrency is saturated, self-hosted RWKV can lower API-equivalent price

Model	Current public API price	Low/moderate RWKV target	High-concurrency RWKV target	Note
DeepSeek V4 Flash	\$0.14 in / \$0.28 out	\$0.10-\$0.13 / \$0.20-\$0.26	\$0.04-\$0.10 / \$0.06-\$0.16	cache-hit input is hard to beat
DeepSeek V4 Pro (flagship case) discounted	\$0.435 / \$0.87	\$0.35-\$0.43 / \$0.65-\$0.85	\$0.15-\$0.35 / \$0.25-\$0.55	needs high utilization
DeepSeek V4 Pro (flagship case) list	\$1.74 / \$3.48	\$1.20-\$1.60 / \$2.40-\$3.20	\$0.40-\$1.00 / \$0.60-\$1.50	can beat list price
Case Anchor B	\$1.40 / \$4.40	\$0.80-\$1.10 / \$2.20-\$3.30	\$0.20-\$0.70 / \$0.60-\$1.80	largest upside

Cost formula

self-hosted \$/MTok = GPU-hour price / (aggregate tok/s * 3600 / 1M) + overhead.

Critical assumption

Not a provider quote. Assumes near-teacher quality and high batch occupancy in serving.

Unit: USD per million tokens, input/output. Recheck live pricing before investment or launch.

Candidate-model choice depends on engineering risk, economic upside and partner objective

Target	Pros	Cons / risks	Best role
Case A: low-risk long-context model	smaller active compute, existing KV optimization, open weights; useful for migration-path validation	must prove RWKV still improves high-concurrency economics over a strong KV baseline	Low-risk case anchor
Case B: high-upside cost model	higher serving cost and clearer long-context/KV pressure; useful for proving commercial upside	larger total-parameter and MoE complexity; needs more infra and evaluation investment	High-upside case anchor
Flagship case: very large model	highest-quality or longest-context target; best after the migration path matures	teacher+student memory, storage and failure cost are severe	Flagship partnership project

Low-risk case

This type of model validates architecture mapping, training stability and serving kernels without overpromising maximum savings.

High-upside case

This type of model is better after the migration path is stable, when the goal is larger commercial cost reduction.

Flagship case

Flagship-scale models are better as joint validation projects after the framework matures.

A successful migration must pass quality, long-context, and serving gates, not only training loss

Gate	Pass standard	Failure signal	Response
Quality parity	core eval >=90%-95% of teacher	knowledge, reasoning or code drops	revisit KL/hidden targets and data
Long context	32K/128K/200K/1M stable by target	NIAH/RULER unusable	long-context KL/CE and state design
Serving gain	high-batch aggregate tok/s and VRAM beat teacher	single-stream works; batch does not scale	kernel/scheduler work
Tools / agents	tool-call, repo-agent and SWE slices do not regress	format drift, long chains fail	GAD/SFT post-alignment
Operability	checkpoint, quant, monitoring, rollback are practical	only runs in research script	serve Flash first, not Pro

Minimum launch gate

Quality parity, long-context validity and high-concurrency cost reduction all need evidence. Missing one means research prototype.

Paradigm-migration gate

The serving stack must turn KV-cache fleet pressure into recurrent-state fleet pressure.

Partner roadmap: establish shared evaluation first, then choose the right full-size migration case



Stage exit condition

If the baseline case misses 90%-95% quality or long-context stability, fix the method before scaling.

Partner pilot

The first partner pilot should use a real high-concurrency agent/coding or long-context workload and measure true batch occupancy.

References support the general migration framework; public models are used as case anchors

No.	Source	Use	
[0]	rwkv.cn	official positioning: next architecture, no KV cache, high concurrency	https://www.rwkv.cn/
[1]	RWKV: Reinventing RNNs for the Transformer Era	RWKV foundation paper	arXiv:2305.13048
[2]	Eagle and Finch: RWKV with Matrix-Valued States and Dynamic Recurrence	matrix state and dynamic recurrence	arXiv:2404.05892
[3]	RWKV-7 "Goose" with Expressive Dynamic State Evolution	expressivity and dynamic state evolution	arXiv:2503.14456
[4]	RADLADS: Rapid Attention Distillation to Linear Attention Decoders at Scale	full-size conversion method	arXiv:2505.03005
[5]	GAD: Generalized Autoregressive Distillation	post-alignment / smaller student route	arXiv:2511.10643
[6]	OpenMOSE RWKV-GLM-4.7-Flash-exp	30B full-size RWKV reference	Hugging Face
[7]	Public model cards and technical reports	size, context, KV/cache and price assumptions	HF / docs
[8]	RWKV high-concurrency tutorial	RTX5090 batch 1/320/960 anchor numbers	mp.weixin local HTML

[9] DPLR time parallelism: explicit transition, chunk-wise affine and prefix-scan.

Partner-facing recommendation

Treat RWKV migration as a joint architecture-paradigm and serving-economics program. GLM/DeepSeek are public case anchors; the actual target should be selected by partner objective.

Note: API pricing, model-card metadata and paper versions can change; recheck before investment or launch.